EE789
Aa Assignment
Hemant Hajare 20D070037

Question 2

prog.aa

```
$storage remainder := $uint<8>
$storage output := $uint<8>
```

These are used to accumulate the quotient in the output storage element and remainder in the remainder storage element. Initially both of them are initialized to 0.

```
a_in, I
```

These are temporary variables in the loop, loopback. They are initialized to a, 0 respectively on entry and assigned next_a, NI from the next iteration. a_in, NI will be updated in the process.

```
$volatile continue_flag := (I < 8)
```

This is the stopping condition for the loop. We right shift the dividend in every iteration to extract the successive MBSs till we reach the LSB. Since this is an 8-bit binary number we iterate through 0 to 7.

```
remainder := (remainder << 1)
remainder := (remainder | a_in [] 7)
```

These are used to left shift the remainder and add the extracted MSB of a_in to the shifted remainder.

```
next_a := (a_in << 1)
```

This left shifts the dividend for the next iteration.

```
$if remainder >= b $then
        remainder := remainder - b
        output := output + (1 << (7-I))
$endif
```

This compares the remainder with divisor b. If the remainder is greater than b we update the remainder with remainder - b and thus add 1 shifted by (7-I) to the output which accumulates the quotient.

NI := I + 1
This updates the iterator I for the next iteration.

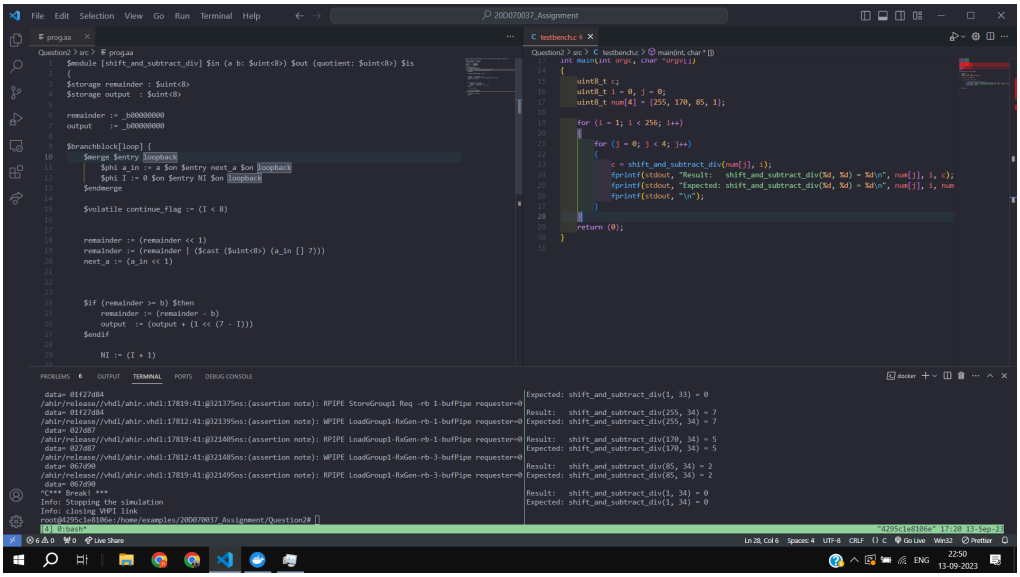
Once we are done iterating through all the bits of the dividend we assign the output to the quotient.

testbench.c
Ideally, the total number of testcases for the testbench should be 256*255 = 65280 (255 because division by 0 is not defined. In my implementation I have assumed that the user will not apply 0 as the divisor). We test only some minimal testcases by giving some rudimentary arguments.

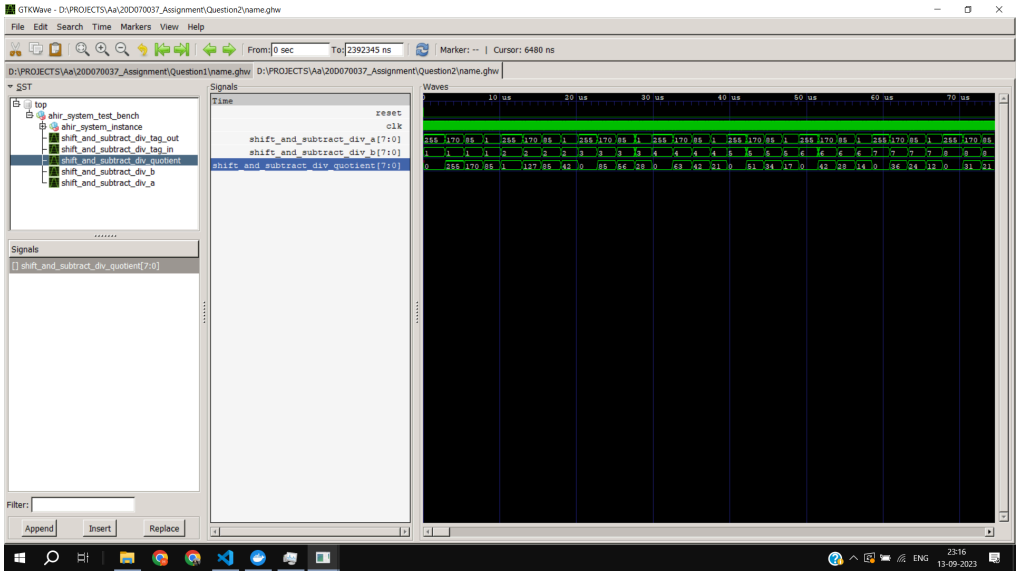
As for our design we take following dividends,

1. 1 1 1 1 1 1 1 1 = 255
2. 1 0 1 0 1 0 1 0 = 170
3. 0 1 0 1 0 1 0 1 = 85
4. 0 0 0 0 0 0 0 1 = 1

Testcase 1. checks the division of the highest possible number with all the numbers. Testcase 2. checks if shifting operation for dividend works correctly at the even places. Testcase 3. checks if the shifting operation for dividend works correctly at the odd places and testcase 4. checks division of unity by all the numbers.



GHDL Simulation shift and subtract divider



GTKWAVE for shift and subtract divider